SLTC Research University

ශ්‍රී ලංකා තාක්ෂණික විශ්වවිද්‍යාලය - පර්යේෂණ

SLTC Research University

Bachelor of Applied Information Technology Degree Programme – (DAIT)

SLTC
Research University

" A wise man listens to advice and corrects his path but the ignorant is drifted by the waves. "

SLTC Research University

DAIT

Module: CIT110

Object Oriented Approach to Programming

python™

SLTC Research University

DAIT

Event Driven Programming & GUI Concepts

Event Driven Programming

Events

An event refers to an action or occurrence that takes place during the execution of a program.

It can be triggered by various other means too. Example: user input, system events, or changes in the program's state.

Events are an essential part of event-driven programming, where the flow of the program is determined by events rather than a linear sequence of instructions.

කල දුටු කල වල ඉහගන්

Event Driven Programming

It is a programming paradigm that deals with the occurrence of events or messages among different components of a system.

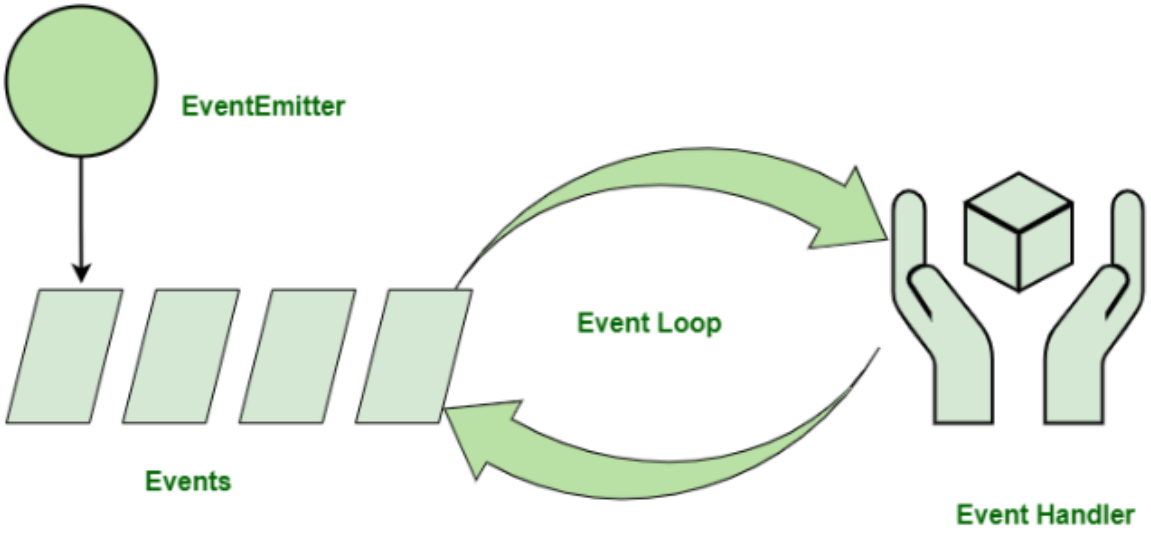
In the Event-driven programming, the program's execution is driven by events, such as user actions, sensor inputs, or messages from other components.

Event-driven programming is commonly used in Graphical User Interfaces (GUIs), where user actions, such as clicking a button or typing on a keyboard, trigger events that are handled by the GUI framework.

Event Driven Programming

There are three actors in EDP namely Event Emitters, Event Listeners and Event Handlers.

When an event occurs, the corresponding event handler is triggered, and it performs the necessary actions or computations.



කල දුටු කල වල ඉහගන්
කැටිය රත්වෙලා රොටිය පුච්චා ගමු

GUIs Programming in Python

SLTC Research University

GUI

GUI stands for Graphical User Interface

GUI is a program that enables a user to communicate with a computer through the use of graphical/visual components.

GUIs are widely used in software development to create visually appealing and user-friendly applications.

Python offers multiple options for developing GUIs. The standard GUI library that comes along with the Python is "tkinter"

Some Other GUI Libraries in Python

1. PyQt5
2. wxPython
3. Kivy (Open-source Multiplatform
4. PyGUI
5. PySimpleGUI
6. Wax
7. Libavg

These libraries (Frameworks) provide a set of tools and widgets that allow software developers to design and build GUI applications with ease.

Among a large number of frameworks available for GUI programming in Python, Tkinter is the only framework that's built into the Python standard library.

Tkinter is cross-platform, so the same code works on Windows, macOS, and Linux.

GUI Programming with Python

A tkinter GUI application can be created simply as follows.

- Import the tkinter module.

  ```
  import tkinter
  ```

  or

  ```
  from tkinter import *
  ```

- Create the main application window.

  ```
  >>> import tkinter
  >>> window = tkinter.Tk()
  ```

  or

  ```
  >>> from tkinter import *
  >>> window = Tk()
  >>> import tkinter as tk
  >>> root = tk.Tk()
  ```

- Add widgets to the main window.

  Add components like buttons, labels etc.

- Call the main event loop.

  ```
  window.mainloop()
  ```

Creating the Main Window

Example

```
import tkinter as tk

window = tk.Tk()

window.mainloop()
```

Import tinker module

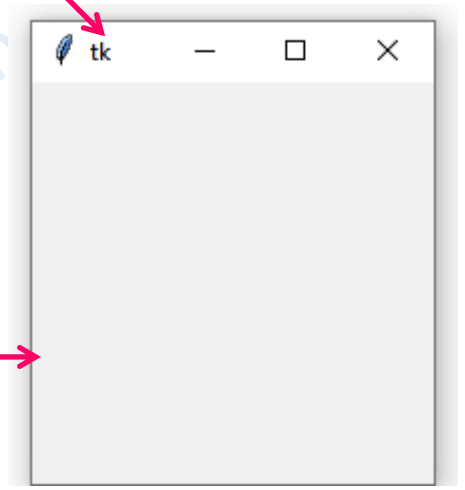
Create main window

Call main event loop

Default Title

Output

Window



- The window that we created is an instance of the Tk class.
- Finally, we start the main event loop using the mainloop method,

Changing the Window Title

SLTC Research University

We will use the title method to set the title of the window.

Example

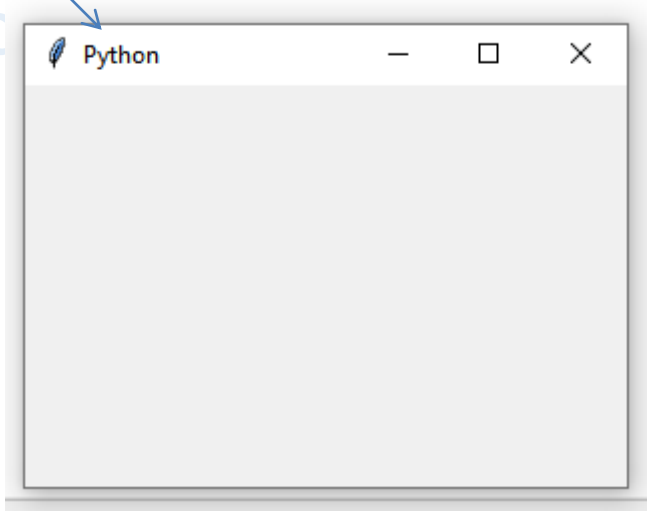
```
from tkinter import *

# Create the main window
window = Tk()

# Set the window title
window.title("Python")

# Start the main event loop
window.mainloop()
```

Title Changed



Changing the Window Size

Example Window size

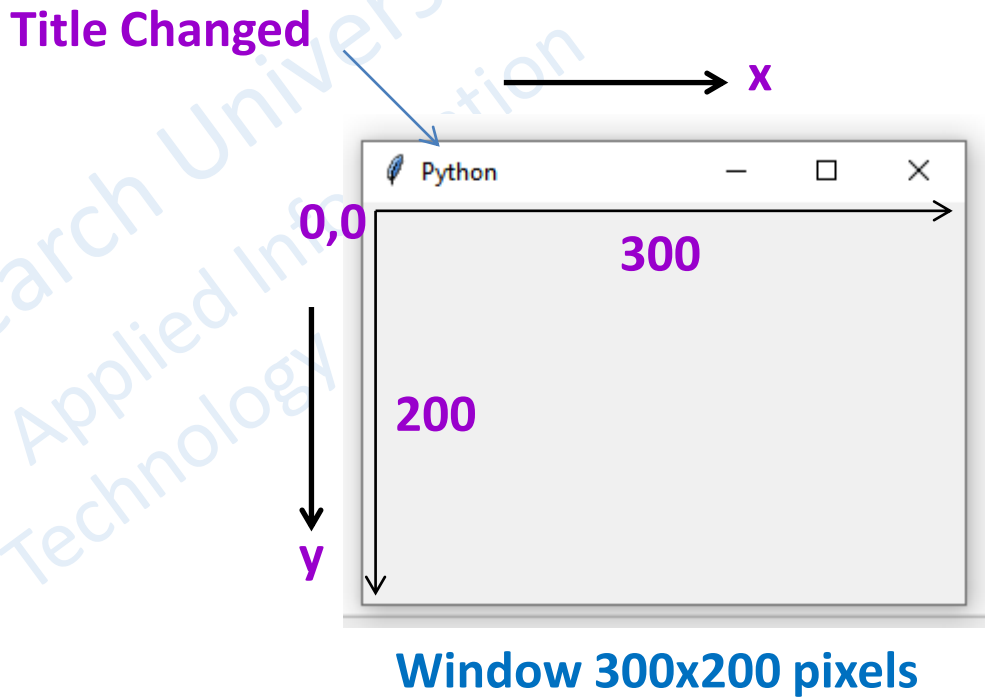
```
from tkinter import *

# Create the main window
window = Tk()

# Set the window title
window.title("Python")

# Set the window size
window.geometry("300x200")

# Start the main event loop
window.mainloop()
```



Window 300x200 pixels

Widgets

Widgets are the building blocks of GUI applications. They are interactive elements that allow users to interact with the application.

They can be labels, buttons, text boxes, checkboxes, radio buttons, sliders, etc that can be placed on the window

Each widget serves a specific purpose and provides a particular functionality.

Widgets can also have various properties and methods that allow you to customize their appearance and behavior. For example, you can set the size, color, and font of a button widget etc.

The following are some of the widgets available in Tkinter

Label	Button	Checkbutton	Entry	Frame
Canvas	Listbox	Menu	Menubutton	Message
Radiobutton	Scale	Scrollbar	Text	Spinbox
LabelFrame	Dialog			

Geometry Configuration Manager (CGM)

The Geometry Configuration Manager (Layout Manager) is a tool used to manipulate geometric configurations. It allows you to control the placement and size of widgets within a window. With the GCM You can modify geometric properties such as points, lines, curves, and shapes.

Types of Geometry Managers

1. **Pack()**

Pack() arranges widgets in a horizontal or vertical manner, stacking them one after another. The pack manager automatically adjusts the size of the widgets based on their content.

Geometry Configuration Manager …

SLTC Research University

2. Grid()

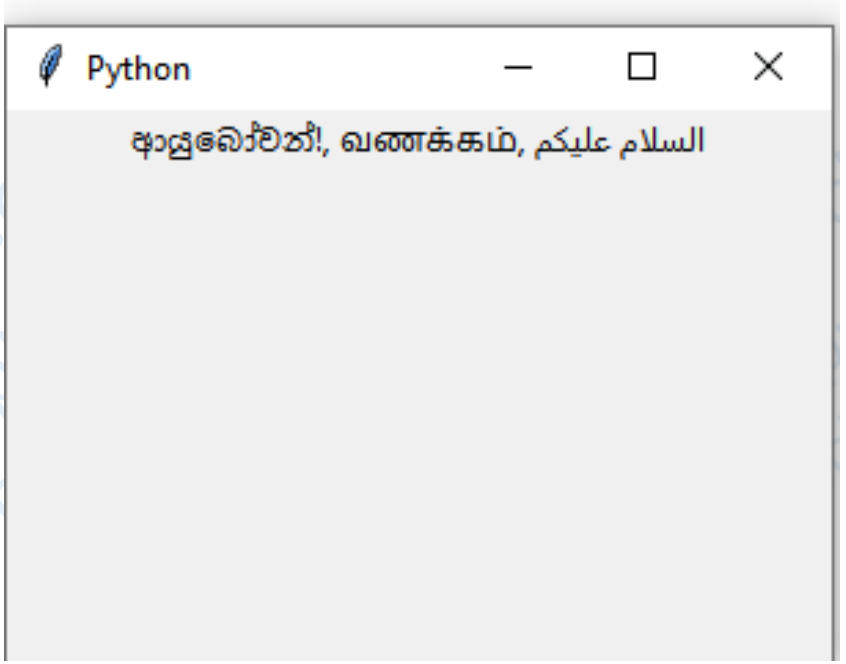
Grid() method takes several parameters, such as row, column, rowspan, columnspan etc. which determine the position and behavior of the widget within the grid.

3. Place()

The "place" manager gives you full control over the placement and size of widgets.

Label Widget

The label widget provides information or instructions to the user. We use Label class in the tkinter module to create a label. A widget used to display text on the screen




```
import tkinter as tk

# Create the main window
window = tk.Tk()

# Set the window title
window.title("Python")

# Set the window size
window.geometry("300x200")

# Create a label widget
lab = tk.Label(window, text="ආයුබෝවන්!, வணக்கம், السلام عليكم")
# Pack the label widget onto the window
lab.pack()

# Run the main event loop
window.mainloop()
```

Using 'padx' and 'pady' in widget options

In Tkinter, padx and pady parameters are used in widget options to control the padding around a widget (padding is the free space).

padx: This is used to specify the horizontal padding around a widget. It determines the amount of extra space to be added on the left and right sides of the widget.

pady: This is used to specify the vertical padding around a widget. It determines the amount of extra space to be added above and below the widget.

Example: Let us construct a colourful row of numbers as given below. Please watch the video.

EXERCISES for the Success

Study the lesson concisely many times before you try the following questions.

(Do not waste bullets)

SLTC Research University

Question: 1

Which one of the following libraries in Python is commonly used for creating GUI applications?

a) Tkinter

b) NumPy

c) Pandas

d) Matplotlib

Question: 2

Which of the following is NOT a benefit of using a GUI?

a) Improved user experience

b) Enhanced visual appeal

c) Increased development complexity

d) Simplified user interaction

SLTC Research University

Question: 3

What is the purpose of an event handler in GUI programming?

a) To handle user input and interactions

b) To define the layout and appearance of the GUI

c) To perform quick calculations

d) To handle object communication

mmja5as4yoaskIobasninTo handle user input and interactions

SLTC Research University

Question: 4

Which one of the following is an example of an event in event-driven programming?

a. A mouse click

b. A loop iteration

c. A variable assignment

d. A function call

Question: 5

What is the purpose of an event handler in event-driven programming?

a. To handle errors and exceptions

b. To define the random structure for a the program

c. To optimize the program's performance

d. To respond to specific events

End of Lesson 8